

WIPRO PYTHON AUTOMATION

REPORT-2

Module 2: Unit Test Frameworks

Name: Debanjali De

Enrollment Number: 12023002001076

Year: 4th Year

Branch: Computer Science and Engineering

Wipro Training Programme, 2026

TABLE OF CONTENTS

1. Introduction
2. Module 2 – Unit Test Frameworks
 - 2.1 UnitTest
 - 2.2 PyTest Framework
 - 2.3 Page Object Model
3. Module Summary
4. Learning Outcomes
5. Conclusion

1. Introduction

Unit testing is a software testing approach in which individual units of software are tested independently to verify that they produce the expected results. A unit may be a function, method, class, or small logical component.

A test framework provides a structured environment for creating, organizing, executing, and reporting tests. It also supports reusable activities such as test setup, teardown, assertions, fixtures, test discovery, and grouping.

Module 2, “Unit Test Frameworks,” introduces Python's unittest framework, the PyTest framework, and the Page Object Model. The module progresses from basic unit-testing concepts to PyTest features and then to a maintainable design pattern for Selenium automation.

2. Module 2 – Unit Test Frameworks

2.1 UnitTest

unittest is Python's built-in unit testing framework. It provides a structured, class-based approach for creating test cases, defining setup and teardown activities, making assertions, and grouping tests into suites.

unittest Introduction

The unittest framework organizes tests around test cases and test methods. A test method performs a specific operation and verifies its expected outcome using an assertion.

- Provides a standard structure for writing and executing unit tests.
- Supports reusable setup and teardown operations.
- Provides multiple assertion methods for validation.
- Allows related tests to be grouped into test suites.

First Test Case

A first test case normally follows three logical stages:

- Arrange – prepare the required data, objects, or test environment.

- Act – perform the operation being tested.
- Assert – compare the actual result with the expected result.

A test case passes when its expected conditions are satisfied and fails when an assertion does not hold.

Implement Class Level SetUp and TearDown Methods

Setup and teardown methods prepare and clean the test environment. Class-level setup is used when resources need to be prepared once for a group of tests, while class-level teardown performs cleanup after the tests in that class have completed.

Stage	Purpose	Typical use
Class-level setup	Prepare shared resources before the class tests run.	Browser/session/configuration setup.
Test method	Execute the individual test scenario.	Action and validation.
Class-level teardown	Release shared resources after the class tests finish.	Closing or cleanup.

Assert a Test Method

An assertion verifies whether an actual result satisfies an expected condition. Assertions are the main mechanism used to determine whether a test has passed or failed.

Assertion concept	Meaning
Equality	Actual value should equal expected value.
True condition	A condition should evaluate to true.
False condition	A condition should evaluate to false.
Membership	A value should be present in a collection or sequence.
Identity	Two references should or should not refer to the same object.

Create a Test Suite

A test suite is a collection of related test cases that can be executed together. Suites are useful when a project contains several tests that belong to the same functional area.

2.2 PyTest Framework

PyTest is a Python testing framework designed to make test development concise and readable while providing powerful features such as automatic discovery, fixtures, parametrization, markers, plugins, and reporting.

Introduction of PyTest

PyTest supports function-based and class-based test organization. It uses naming conventions to discover test files and test functions automatically, reducing the amount of framework boilerplate required.

Installing PyTest and Naming Conventions

PyTest is installed into the Python environment used for the project. Naming conventions are important because they determine which files, classes, and functions are automatically discovered as tests.

Item	Common convention	Purpose
Test file	test_*.py or *_test.py	Allows automatic test discovery.
Test function	test_*	Identifies a function as a test.
Test class	Test*	Identifies a class that contains tests.
Test directory	Project-defined	Provides an organized location for test files.

Test Files, Test Methods and Assertions

A pytest test file contains test functions or test classes. Each test method represents a behavior or condition that needs to be verified. pytest commonly uses Python's assert statement for direct and readable validation.

Subtests and Test Utilities

Test utilities are reusable supporting components used for activities such as test-data preparation, configuration, common operations, logging, screenshots, or other repeated tasks. They help keep test cases focused on the behavior being verified.

Subtests or grouped checks can be used when several related validations belong to one broader test scenario. The exact implementation depends on the testing framework and project design.

Fixtures and conftest.py

A fixture is a reusable pytest component that prepares data, resources, or an environment for one or more tests. Fixtures reduce duplicated setup code and can be reused across multiple test functions.

The conftest.py file is a special pytest configuration/support file commonly used to define shared fixtures and hooks. Fixtures defined there can be discovered by pytest without requiring direct imports in every test.

pytest Features

- Automatic test discovery.
- Simple and readable assert-based validation.
- Reusable fixtures.
- Parametrization for running the same test with multiple data sets.
- Markers for categorizing and selecting tests.
- Plugin support for extending pytest.
- Command-line execution and test selection.
- Integration with reporting tools.

Run Multiple Test Cases

pytest can execute multiple test cases by targeting a test file, directory, or selected test objects. Tests can also be selected or grouped using naming conventions, markers, and command-line options.

Moving Selenium Unit to pytest

Selenium WebDriver is responsible for browser interaction, while pytest can provide the test execution structure. Moving Selenium tests into pytest allows browser tests to use pytest discovery, fixtures, assertions, parametrization, markers, and reporting.

Selenium responsibility	pytest responsibility
Launch and control the browser.	Discover and execute tests.
Locate and interact with web elements.	Organize test functions/classes.
Navigate and perform UI actions.	Provide assertions and fixtures.
Read browser state.	Provide selection, parametrization, and reporting support.

Install pytest HTML Plugin and Generate HTML Reports

pytest can be extended through plugins. An HTML reporting plugin can produce a browser-viewable report containing execution information such as test names, results, and failure details. HTML reports make test execution results easier to review and share.

2.3 Page Object Model

The Page Object Model (POM) is a design pattern used in UI automation to represent application pages or page components as programming objects. It separates page interaction logic from test scenarios and helps reduce duplicated locator and interaction code.

What is Page Object Model?

In POM, a page class represents a page or component of the application. It contains the locators and methods required to interact with that page. Test classes focus on test scenarios and validations while calling the page-object methods.

Understanding Base Class, Page Class, Test Class, Utility Classes, Configuration

Component	Definition / role
Base Class	Provides reusable foundation behavior shared by other framework classes.
Page Class	Represents a page/component and contains its locators and interaction methods.
Test Class	Contains test scenarios, execution flow, and validations.
Utility Class	Contains reusable helper operations used across the framework.
Configuration	Stores or manages environment, browser, URL, or test-related configuration.

Page Object Model Structure

A typical POM framework separates page objects, test classes, base components, utilities, test data, and configuration. This separation creates a clear division of responsibility.

Layer	Main responsibility
Page Object	Element locators and page-level actions.
Test Layer	Business/test scenario and validation.
Base Layer	Common reusable framework behavior.
Utility Layer	Reusable helper functions.
Data / Configuration	Test inputs and environment settings.

Base Page and Utility Concept

A Base Page contains common operations that can be shared by several page classes. Utility components provide reusable functions for activities such as data reading, screenshots, waits, configuration handling, or other common framework tasks.

Inheriting Base Page Class

Inheritance allows a specific page class to reuse functionality from a Base Page. Common operations can therefore be implemented once and reused by multiple page classes, reducing duplication and improving maintainability.

Utility Read/CSV Data

Automation tests often use external data such as usernames, passwords, expected values, or test inputs. A utility component can centralize the reading and processing of CSV data so that test cases remain focused on the scenario rather than data-access details.

Convert Test Case to Page Object Model Framework

Converting a test case into a POM framework involves identifying the application pages, moving page-specific locators and interactions into page classes, keeping scenario flow and assertions in the test layer, and reusing common functionality through base and utility classes.

The main benefits are reduced duplication, easier maintenance, improved readability, and better separation of responsibilities.

Taking Screenshots on Test Failure

A screenshot captures the visual state of the application at a particular point in time. Capturing a screenshot when a test fails provides useful evidence for troubleshooting and helps identify the browser state that caused the failure.

3. Module Summary

Module 2 introduces the foundations of structured test automation. unittest provides Python's built-in class-based testing approach with test cases, setup/teardown, assertions, and suites. pytest provides a more flexible testing framework with discovery, fixtures, parametrization, plugins, command-line execution, and reporting. The Page Object Model then provides a maintainable architecture for organizing Selenium UI automation.

Topic	Key concepts
unittest	Test cases, setup/teardown, assertions, suites.
pytest	Discovery, naming, assertions, fixtures, conftest.py, features, execution, plugins, HTML reports.
Page Object Model	Page classes, test classes, base classes, utilities, configuration, inheritance, data, screenshots.

4. Learning Outcomes

- Define unit testing and explain the purpose of a unit-test framework.
- Explain unittest test cases, class-level setup/teardown, assertions, and test suites.
- Explain pytest installation, naming conventions, test discovery, and assertions.
- Understand pytest fixtures, conftest.py, utilities, features, and multiple-test execution.
- Explain how Selenium automation can be organized and executed using pytest.
- Understand the purpose of pytest plugins and HTML test reporting.
- Define the Page Object Model and explain its role in maintainable UI automation.
- Differentiate Base Class, Page Class, Test Class, Utility Classes, and Configuration.
- Understand inheritance, CSV/test-data utilities, test-case conversion to POM, and failure screenshots.

5. Conclusion

Unit test frameworks provide the structure required for reliable and maintainable automated testing. unittest establishes the basic concepts of test cases, setup, assertions, and suites. pytest expands these capabilities through fixtures, discovery, parametrization, plugins, and reporting. The Page Object Model complements Selenium automation by separating page interactions from test scenarios. Together, these topics form an important foundation for structured Python and Selenium test automation.